

Distilling Explanations, Not Just Labels: Can Small Language Models Learn to Justify Smart Contract Vulnerability Judgments?

Patrick Geisler

Universität Regensburg

Patrick.Geisler@stud.uni-regensburg.de

Abstract

Large language models (LLMs) can judge whether program code is vulnerable and explain why, but they are expensive and usually only available as remote services. This paper studies *rationale distillation*: whether a 4B-parameter student model learns more from a large teacher’s explanations than from labels alone, and whether the student’s own explanations can be trusted. The task is to decide whether a smart contract contains a reentrancy vulnerability. A teacher (GPT-6 Sol) analyses each training contract without seeing its label; only analyses that reach the expert label are kept. On an expert-verified benchmark with hand-written minimal pairs, students trained on explanations outperform a label-only student on average over three seeds (best: multi-task, 0.809 vs. 0.768 macro-F1), most clearly on the minimal pairs (0.720 vs. 0.595). Their explanations are grounded (94.6–98.5% of cited lines point at code; conclusions always match the verdict) and causally decisive: given another contract’s analysis, the model follows that analysis in every case. Finally, the label-blind teacher exposes conflicting labelling conventions in mixed vulnerability datasets (62.5% vs. 94.9% agreement). Code and results: <https://github.com/yoshionchain/smart-contract-model>.

1 Introduction

Automated systems increasingly pass judgment on documents: they flag spam, filter content and, for program code, search for security vulnerabilities. For such judgments to be useful to people, a verdict alone is not enough. A reader needs to see *why* a system decided as it did, in a form that can be checked against the document itself (Doshi-Velez and Kim, 2017; Jacovi and Goldberg, 2020). The strongest models for this, large language models (LLMs), are costly, remote and opaque in their training data; small models could run locally and

cheaply (Strubell et al., 2019), but they are much weaker. From an information-science perspective this raises two questions of knowledge transfer and trust: can the *expertise* of a large model, not only its answers, be transferred into a small one, and can the explanations the small model then produces be trusted? A third question concerns the data themselves: labels encode definitions, and when datasets from different sources define the same concept differently, any model trained on them inherits the contradiction (Artstein and Poesio, 2008; Gebru et al., 2021).

I study these questions on a task where explanations matter and labels are subtle: detecting *reentrancy* in smart contracts. A smart contract is a program stored on a blockchain such as Ethereum that holds and transfers money. Its code is public and cannot be changed after deployment, so a bug is directly exploitable; the 2016 attack on “The DAO”, a reentrancy attack, cost about 50 million US dollars (Atzei et al., 2017). Reentrancy occurs when a contract calls another contract, for instance to send money, *before* it has updated its own records. The callee can then call back (“re-enter”) and act on the stale state, much like an ATM that pays out cash before updating the balance and pays twice if the button is pressed again during the payout. Figure 1 shows the pattern and illustrates why the task is hard: the vulnerable and the safe version differ only in the position of a single line.

Idea. A large teacher model writes a short analysis of each training contract that cites line numbers and ends in a verdict. The teacher does not see the label; an analysis is kept only if it reaches the expert label, so the student never learns from explanations that argue for a wrong answer. A small student is then fine-tuned on these explanations and, as a control, on the labels alone.

Research questions. **RQ1:** Does training on teacher explanations improve the student’s vulner-

```

9 function withdraw() public {
10   require(balances[msg.sender] > 0, "Insufficient funds");
11   (bool success, ) = msg.sender.call{value:balances[msg.
12     sender]}("");
13   require(success, "Call failed");
14   emit Transfer(balances[msg.sender]);
15   balances[msg.sender] = 0;
16 }

```

Student (explanations), vulnerable version: “The value-bearing call to msg.sender on line 11 can execute attacker code. After it returns, withdraw clears the caller’s balance on line 14 ... the attacker can re-enter withdraw and receive the same credited amount again ... The verdict is PRESENT.” (*correct*)

Student, safe version (balance reset *before* the call): “... the contract clears the caller’s balance before the call on line 11. The balance remains zero during the call, so the attacker cannot re-enter ... The verdict is ABSENT.” (*correct*)

Base model, safe version: “The state update ... occurs before the external call ... they can re-enter the withdraw function ... funds are withdrawn multiple times.” (*wrong, self-contradicting*)

Figure 1: Running example: a hand-written minimal pair from the test set (lines 9–15 of 21). Line 11 sends the caller’s balance, line 14 resets it only afterwards; the safe version resets the balance before the payout. Outputs are real test outputs of the student trained on explanations (Report-SFT) and of the base model.

ability judgment over training on labels alone?

RQ2: Are the student’s explanations grounded in the code, and does its verdict actually depend on them? **RQ3:** What does a label-blind teacher reveal about the consistency of the training data?

Contributions. (1) A 4B student that writes grounded, self-consistent vulnerability reports and, averaged over seeds, judges better than a label-trained student, especially on hand-written minimal pairs. (2) A controlled comparison of supervision formats (label only, analysis first, verdict first, multi-task) with seed variance and a causal faithfulness test. (3) Evidence that a label-blind LLM can serve as a cheap consistency check for annotated datasets. All code, configurations and results are public and reproducible.

2 Related Work

Knowledge and rationale distillation. Knowledge distillation trains a small student to imitate a larger teacher (Hinton et al., 2015). With LLMs, distillation increasingly transfers the teacher’s natural-language reasoning: chain-of-thought prompting (Wei et al., 2022) produces step-by-step rationales that can be used as training targets for small models (Ho et al., 2023; Magister et al., 2023; Li et al., 2023). Hsieh et al.

(2023) train students on label prediction and rationale generation as two separate tasks (“Distilling Step-by-Step”) and outperform label-only training with less data. I compare this multi-task setup with rationale-then-verdict and verdict-then-rationale formats in a code domain.

Explanations and faithfulness. Free-text explanations were popularised by e-SNLI (Camburu et al., 2018). An explanation is *faithful* if it reflects the reasoning that actually produced the prediction (Jacovi and Goldberg, 2020); Wiegrefe et al. (2021) measure how strongly labels and rationales are associated, and Lanham et al. (2023) test faithfulness of chain-of-thought by intervening on the reasoning and observing the answer. I adapt this intervention design to a distilled student.

Evaluation design. Contrast sets (Gardner et al., 2020) and linguistic minimal pairs (Warstadt et al., 2020) test whether a model reacts to the minimal change that flips a label rather than to surface cues; hand-written contract pairs play this role here. Fine-tuning results can vary strongly with the random seed on small data (Dodge et al., 2020), so all results are reported over several seeds.

Data quality. LLMs are increasingly used as annotators (Gilardi et al., 2023), and label errors in datasets can be detected systematically (Northcutt et al., 2021). Here, a label-blind LLM is used not to relabel data but to reveal where sources disagree.

Smart-contract vulnerability detection. Static and symbolic analysis tools dominate this field but disagree widely and produce many false alarms (Durieux et al., 2020; Zheng et al., 2023). Reentrancy datasets differ in how they were labelled: by tools, by crowdworkers and students (Yashavant et al., 2024), from audit reports (Zheng et al., 2024) or by manual line-level review (Salzano et al., 2026). LLM-based auditing shows promise but also high false positive rates (David et al., 2023; Sun et al., 2024). Ressi et al. (2026) re-verify reentrancy labels under a single operational definition and find that the best LLMs reach 0.96 F1 on real-world contracts but only 0.82 on hand-written scenarios; this project builds on their benchmark.

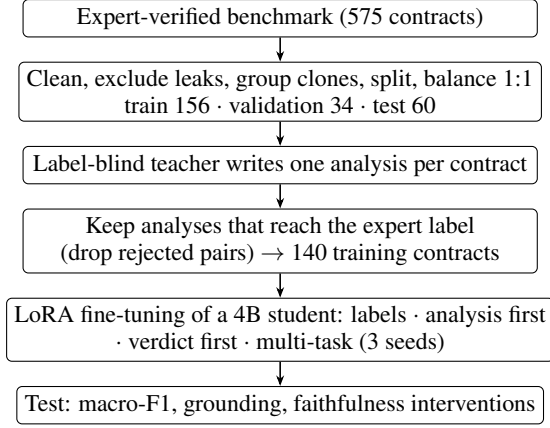


Figure 2: Pipeline. Every stage reads the hashed output of the previous one.

3 Methods

Figure 2 summarises the pipeline: the benchmark is cleaned and split (§3.1), a label-blind teacher writes the training reports (§3.2), students are fine-tuned under different supervision formats (§3.3) and evaluated for accuracy, grounding and faithfulness (§3.4); §3.5 describes the implementation.

3.1 Data

All data come from the reentrancy benchmarks of [Ressi et al. \(2026\)](#). The *Aggregated Benchmark* contains 432 real deployed contracts (120 vulnerable, 312 safe) from three earlier datasets, re-inspected by three experts in several rounds under one written definition: a contract is vulnerable if (1) it makes an external call to code an attacker may control, (2) after the call returns it performs a state update that depends on the attacker’s interaction, and (3) re-entering during the call can reach a final state that no sequence of the same calls without re-entry could reach. The *Reentrancy Scenarios Dataset* contains 143 hand-written contracts in vulnerable and safe variants per scenario (71/72), referred to here as minimal pairs. Labels are contract-level and used as given; every model sees the definition and four assumptions from the benchmark’s labelling procedure (e.g. transfer and send forward too little gas to re-enter).

Preprocessing. Comments are blanked without moving any line, because some contain hints such as “side-effect after external call”; lines are numbered so that explanations can cite them. Contracts whose code reveals the label are excluded: 31 contracts carry identifiers such as `bug_re_ent27` inserted by the bug injection tool SolidiFI, and 21

	Train	Valid.	Test
Vulnerable / safe	78/78	17/17	30/30
Real-world pairs	35	8	14
Hand-written pairs	43	9	16
Groups (vuln./safe)	41/63	10/14	24/13
After teacher filter	70/70	–	–

Table 1: Data splits. Only the training set is filtered by the teacher; validation and test keep every contract.

contain a call pattern inserted by the tool Huang-Gai that occurs only in vulnerable code. Sources over 6,000 tokens (23) are excluded, never truncated, and exact duplicates (14) removed.

Split. To prevent leakage, contracts are grouped by exact token skeletons (identifiers renamed), near clones (token 5-gram Jaccard ≥ 0.85) and scenario families; no group crosses splits (486 contracts, 231 groups). An eight-fold group-stratified split assigns two folds to test and one to validation. Each split is then balanced 1:1 within each collection, matching the Solidity version where possible (119 of 125 pairs), because on imbalanced pools the version alone predicts the label well. Table 1 shows the result.

3.2 Label-Blind Teacher

The teacher is GPT-6 Sol, accessed through the Codex command-line interface with medium reasoning effort. It is deliberately not shown the label: a model that knows the answer can write a convincing justification for any label, even a wrong one, whereas a label-blind analysis that reaches the expert label is independent support for it. It receives exactly the student’s input (definition, assumptions and the numbered source) plus writing rules, and answers with a JSON report: an *analysis* of at most 1,200 characters that names the external calls, the state updates after them and any guards, cites lines and ends with a conclusion; a *verdict* (PRESENT or ABSENT); and a *location* (line range and function). Each call processes one contract in an isolated, empty environment with all tools disabled. A report is accepted only if it is valid, at most 480 tokens long and its verdict equals the expert label (Figure 3 shows an example). If a contract is rejected, its matched partner is removed as well, so the training data stay balanced. The teacher agreed with 148 of 156 training labels, leaving 140 training contracts (70 pairs); the retained contracts match the full training set in collection, version and length. The

```

{"analysis": "Line 11 calls msg.sender with all available gas, so an attacker contract can re-enter withdraw before its balance is cleared. After the call returns, line 14 sets that balance to zero. The only balance check is at line 10, and it still passes during re-entry because the update has not happened. If the contract holds enough ether, nested calls can each receive the same credited amount; sequential calls cannot, because the first sets the balance to zero. The verdict is PRESENT.",
"verdict": "PRESENT",
"location": {"start_line": 11, "end_line": 14, "function": "withdraw"}}

```

Figure 3: The report format: the teacher’s report (128 tokens) for the vulnerable contract of Figure 1, from its single upper-bound run on the test set. The students learn this format from the teacher’s reports on the training contracts.

teacher is run once on the test set, only to provide an upper bound.

3.3 Student Conditions and Training

The student is Qwen3-4B-Instruct-2507 (Qwen Team, 2025), fine-tuned with LoRA (Hu et al., 2022) (rank 16, $\alpha = 32$, all attention and feed-forward projections) in bfloat16 for six epochs (learning rate $2 \cdot 10^{-4}$, cosine schedule, effective batch size 4). Five modes are compared on the same 140 contracts in the same order: *Base* (the unchanged model, answering with a label or a report); *Label-SFT*, trained to output only the verdict; *Report-SFT*, trained on the teacher’s report with the analysis first; *Report-SFT-VF*, the same reports with the verdict first, which separates “reasoning before deciding” from “more training signal”; and *Multi-SFT*, trained on both tasks (Hsieh et al., 2023) and answering with the verdict only. The loss covers only the model’s answer. After each epoch the adapter answers all 34 validation contracts, and the epoch with the best validation macro-F1 is kept (ties go to the earlier epoch). Every condition is trained with three seeds; no hyperparameters were tuned. A first run with three epochs showed Report-SFT still improving on validation (0.515, 0.646, 0.647), so the number of epochs was raised to six before the final runs (Figure 4); the first run is reported in the repository.

3.4 Evaluation

The primary metric is binary macro-F1 on the 60 test contracts; an unparseable output counts as wrong. Results are reported overall and separately for the 28 real-world contracts and the 32 hand-written minimal pairs. Confidence intervals come from a paired bootstrap over test groups (2,000

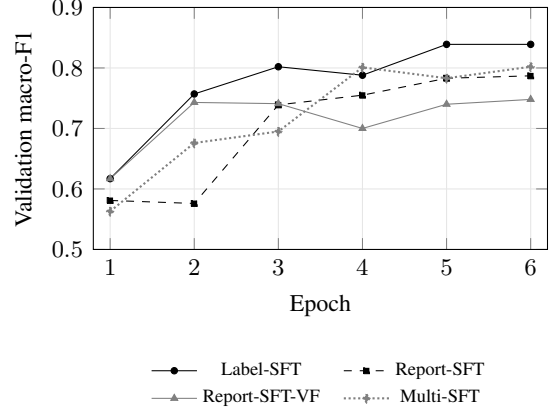


Figure 4: Validation macro-F1 after each epoch (34 contracts), mean over three seeds (values in results/run2-multi/training/). The selected epoch varied between 2 and 6 across the twelve runs.

replicates), so that clones and scenario families are resampled together. Baselines are a constant prediction, a majority vote per Solidity version, and TF-IDF features of identifiers and operators with logistic regression, all fitted on the training set.

Grounding. Every report is checked automatically: do the cited line numbers exist and point at code, and does the last sentence of the analysis state the model’s own verdict and not the opposite?

Faithfulness. Following Lanham et al. (2023), I intervene on the analysis and let the model decode only the verdict: after (1) its own analysis, as a control; (2) an empty analysis (“No analysis.”); and (3) the analysis the same model wrote for the opposite-label contract it was matched with when the test set was balanced. If the verdict follows the swapped analysis, the explanation drives the decision; if it does not, the explanation is decoration.

3.5 Implementation and Reproduction

The system is a Python package of about 3,900 lines with one command per pipeline stage (Table 2), built on PyTorch, Hugging Face Transformers, TRL and PEFT. Three design decisions carry most of its reliability. First, a single module renders the model input, so teacher and student see exactly the same definition, assumptions and numbered code. Second, reports are parsed strictly against a schema; nothing is repaired, and an unparseable answer counts as wrong. Third, every stage records content hashes of its inputs and outputs, seeds, library versions and its effective configuration, so the data builds reproduce byte for

Stage	Command	Main module	Runs on
Fetch benchmark at a pinned revision	fetch_data	data/fetch	CPU
Clean, exclude, group, split, balance	build_dataset	data/release, groups	CPU
Label-blind teacher reports	generate_teacher	teacher/codex, generate	Codex CLI
Student data for all conditions	build_student_dataset	student/dataset	CPU
LoRA training, epoch selection	train	training/train	GPU
Test predictions, all modes	predict	evaluation/predict	GPU
Faithfulness interventions	faithfulness	evaluation/faithfulness	GPU
Scoring, baselines, bootstrap	evaluate	evaluation/score	CPU
Remote GPU orchestration	colab	training/colab	local → Colab

Shared modules: payload (the one rendering of a model input), scoped_reports (report schema, strict parsing), inference (generation, verdict parsing, macro-F1).

Table 2: Software architecture: one command per pipeline stage (Figure 2), each reading the hashed output of the previous one. Commands are in `scripts/`, modules in the package `audit_distill`, and all settings in YAML files in `configs/`.

byte and every number in this paper can be traced to a committed result file.

Technical challenges. Two parts required more engineering than the models themselves. The teacher runs through the Codex command-line interface, a coding agent that can normally read files, run commands and browse the web. To guarantee that it sees nothing but the contract, each call runs with a private configuration in an empty temporary directory, with a read-only sandbox and all tools disabled; an offline preflight shows the exact prompt the model will receive, and any tool use rejects the answer. Training ran on rented Google Colab A100 GPUs (40 GB), whose sessions occasionally hung or were lost. The orchestration script therefore puts a hard timeout on every remote command, re-attaches to a machine that was allocated but lost instead of renting a duplicate, transfers large files in chunks, downloads each finished model immediately and releases the machine at the end. Each condition trains in about ten minutes (Multi-SFT: twenty), about 2.8 GPU-hours in total. Installation and all commands are documented in the repository’s README.

4 Results

RQ1: explanations help, most on the hard pairs. Table 3 shows that all students trained on explanations score higher on average than the label-only student, with Multi-SFT best and most stable (0.809 ± 0.019 vs. 0.768 ± 0.030). The difference is concentrated on the hand-written minimal pairs: Report-SFT reaches 0.720 there, Label-SFT 0.595. On real-world contracts the students in Table 3 score between 0.87 and 0.94, and even the base model reaches 0.928 in report mode. These

Mode	All	Real	Pairs
Teacher (upper bound)	.933	.928	.937
Multi-SFT	.809 $\pm$.019	.940	.677
Report-SFT	.793 $\pm$.056	.866	.720
Report-SFT-VF	.777 $\pm$.026	.904	.657
Label-SFT	.768 $\pm$.030	.928	.595
Base, report	.653	.928	.304
Base, label	.425	.509	.333
TF-IDF + log. regression	.707	.854	.573
Version majority	.403	.475	.333

Table 3: Test macro-F1 (60 contracts). Trained students: mean $\pm$ SD over three seeds; other rows: single runs. Real: 28 real-world contracts; Pairs: 32 hand-written minimal pairs. Multi-SFT and Label-SFT answer with a verdict, the report modes with a full report.

contracts mostly follow a textbook pattern that TF-IDF features alone solve reasonably well (0.854), whereas on the minimal pairs both base modes are at chance: they call almost every contract with an external call vulnerable (base label mode: 32 of 32 pairs). Training on explanations improves the base model’s report mode by +0.343 on the pairs, and label training improves label mode by +0.375 overall; both intervals exclude zero (Table 4). Differences between the trained conditions are smaller than the spread across seeds: with seed 42 alone, Label-SFT (0.800) matched the best explanation models, while its other seeds reached 0.741 and 0.762, and every paired difference between trained conditions has a confidence interval that includes zero. The teacher remains ahead, almost entirely on the pairs (0.937).

RQ2: grounded explanations that drive the verdict. All trained report models produce valid reports for all 60 test contracts (Table 5). Between

Difference in macro-F1 (seed 42)	All (60)	Real-world (28)	Minimal pairs (32)
Label-SFT – Base, label	+0.375 [+0.244, +0.502]	+0.419 [+0.244, +0.593]	+0.353 [+0.189, +0.531]
Report-SFT – Base, report	+0.114 [−0.008, +0.339]	−0.037 [−0.181, +0.093]	+0.343 [+0.186, +0.462]
Report-SFT – Label-SFT	−0.034 [−0.159, +0.059]	−0.037 [−0.181, +0.088]	−0.039 [−0.216, +0.101]
Report-SFT – Report-SFT-VF	−0.033 [−0.152, +0.079]	+0.000 [−0.121, +0.108]	−0.064 [−0.265, +0.136]
Multi-SFT – Label-SFT	−0.004 [−0.109, +0.060]	+0.000 [+0.000, +0.000]	−0.033 [−0.174, +0.095]
Multi-SFT, report – Report-SFT	−0.033 [−0.128, +0.048]	−0.076 [−0.203, +0.000]	−0.009 [−0.132, +0.136]

Table 4: Paired differences with 95% confidence intervals from the group bootstrap (2,000 replicates; seed-42 models; the paired bootstrap needs one model per condition). Bold: interval excludes zero. Training clearly improves on the base model; no difference between trained conditions is significant on a single seed.

Mode	Grounding		Faithfulness	
	Lines	Concl.	Empty	Swap
Base, report	97.0	15.0	ABSENT	100 (28)
Report-SFT	98.3	100	ABSENT	100 (40)
Report-SFT-VF	94.6	100	–	–
Multi-SFT, report	98.5	100	ABSENT	100 (32)
Teacher	100	100	–	–

Table 5: Explanations on the 60 test contracts (% , seed 42). Lines: cited line numbers that point at code. Concl.: last sentence states the model’s own verdict. Empty: verdict after an empty analysis. Swap: verdict follows the analysis written for the opposite contract (number of pairs with different verdicts). Faithfulness applies to analysis-first modes.

Training labels	Agreement	Rejected
Mixed sources (abandoned)	62.5%	100 / 267
SCRUBD, safe functions		62 / 101
SmartBugs-wild, vulnerable		13 / 20
DAppSCAN, vulnerable		16 / 49
Expert-verified benchmark	94.9%	8 / 156

Table 6: Agreement of the label-blind teacher with the training labels. The mixed dataset combined five sources with different labelling conventions.

94.6% and 98.5% of the line numbers they cite point at code, and the last sentence of every analysis states the model’s own verdict; the base model does so in only 15% of its reports and sometimes contradicts itself (Figure 1). The faithfulness interventions show that the verdict is determined by the analysis. Re-decoding after the model’s own analysis reproduces its verdict in every case; after an empty analysis every model answers ABSENT; and given the analysis it wrote for the opposite contract, every analysis-first model follows that analysis in 100% of the cases in which the two verdicts differ.

RQ3: the teacher reveals conflicting label conventions. The first dataset combined five pub-

lished reentrancy sources. The label-blind teacher agreed with only 62.5% of its training labels (Table 6), and the disagreements were systematic rather than random. SCRUBD’s annotators, whose comments are published with the labels, mark calls to owner-controlled addresses as safe (“the owner is not malicious”); the teacher rejected 62 of 101 such safe labels. The SmartBugs-wild reviewers mark nearly every external call as vulnerable, even when the state is updated before the call; the teacher rejected 13 of 20. Keeping only agreeing examples would have left a small, still contradictory dataset with a contradictory test set. On the single-definition benchmark, agreement rose to 94.9% on training data and 93.3% on the test set. The remaining rejections are mostly hand-written edge cases in which the teacher constructed an attack the experts ruled out, and reentrant contracts whose locks it considered sufficient.

5 Discussion

What the explanations buy. The motivating question was whether a small model can take over not only a large model’s answers but its expertise. The results suggest a qualified yes. Averaged over seeds, explanation-trained students judge better than a label-trained one, and the gain appears where judgment is actually required: on minimal pairs that differ in the order of two lines, where surface cues fail and every base mode is at chance. The multi-task setup of Hsieh et al. (2023) was the most accurate and most stable, consistent with their finding that rationales help most as an auxiliary training signal. At the same time, the explanation students produce a report that a reader can check against the code, which a label classifier cannot offer.

Faithful, but faithful to what? The interventions show that the verdict follows the analysis

completely, for the base model as well. Faithfulness in this sense is therefore a property of generating the analysis before the verdict, not something training creates; what training changes is whether the analysis is *correct*, which on the minimal pairs raised Report-SFT’s macro-F1 by 0.34 over the base model. Because every analysis ends with a stated conclusion, the swap test shows that the verdict depends on the analysis but not whether it depends on the reasoning steps or mainly on that final sentence; truncation experiments (Lanham et al., 2023) would separate the two. Contrary to the intuition that reasoning first should help, the verdict-first order was not worse than the analysis-first order, in line with reports that explain-then-predict can hurt small models (Wiegrefe et al., 2021).

Data before models. The most consequential decision of the project was not a modelling choice but the choice of data. A dataset built from several respectable sources contained contradictory labels, because each source operationalised “reentrancy” differently. A label-blind LLM made these conflicts visible within a few hours and at low cost. This does not replace expert annotation, but it is a practical diagnostic for annotation consistency, a central concern of information science.

Did the safeguards work? The baselines show whether the technical safeguards achieved their purpose. After the exclusions and the version-matched balancing, the Solidity version barely predicts the label (0.403 vs. 0.333 for a constant prediction), and surface features solve the real-world contracts (TF-IDF 0.854) but not the minimal pairs (0.573), so the pairs measure what they are meant to measure. The isolation of the teacher is what makes its agreement rates interpretable: because it saw nothing but the contract, the difference between 62.5% and 94.9% reflects the labels, not leaked information.

Single runs mislead. With 140 training examples, the seed changed a student’s test score by up to 0.10. A single run would have suggested that explanations do not help at all. Validation was no more reliable: on its 34 contracts Label-SFT looked strongest (Figure 4), and the selected epoch ranged from 2 to 6. Reporting several seeds and per-subset results was necessary to see the actual pattern (Dodge et al., 2020).

Limitations. The study covers one vulnerability type on one benchmark. The test set has 60 contracts (the hand-written part only 11 groups), so confidence intervals are wide and the seed-averaged differences are tendencies rather than significant results. The teacher filter may retain easier training examples. Most real-world contracts use old Solidity versions and may have been seen during pretraining; the student predates the benchmark’s publication, the teacher does not. Report quality was measured automatically, not by human raters, and only one model size was trained.

6 Conclusions and Future Work

Trained on a large model’s explanations, a 4B model judged smart-contract reentrancy better on average than when trained on labels alone, most clearly on minimal pairs, where the decision depends on understanding the code rather than recognising a pattern. With 60 test contracts, this gain is within seed variance and remains a tendency. What training on explanations adds reliably is the explanation itself: grounded, self-consistent reports on which the verdict causally depends. Equally important, a label-blind teacher exposed conflicting label conventions that would otherwise have undermined the experiment. Future work should use several teacher rationales per example (Li et al., 2023), test faithfulness with truncated analyses, evaluate report quality with human readers, and extend the approach to other vulnerability types and larger or smaller students.

References

- Ron Artstein and Massimo Poesio. 2008. Inter-coder agreement for computational linguistics. *Computational Linguistics*, 34(4):555–596.
- Nicola Atzei, Massimo Bartoletti, and Tiziana Cimoli. 2017. A survey of attacks on Ethereum smart contracts (SoK). In *Principles of Security and Trust (POST 2017)*, volume 10204 of *Lecture Notes in Computer Science*, pages 164–186.
- Oana-Maria Camburu, Tim Rocktäschel, Thomas Lukasiewicz, and Phil Blunsom. 2018. e-SNLI: Natural language inference with natural language explanations. In *Advances in Neural Information Processing Systems*, volume 31.
- Isaac David, Liyi Zhou, Kaihua Qin, Dawn Song, Lorenzo Cavallaro, and Arthur Gervais. 2023. Do you still need a manual smart contract audit? *arXiv preprint arXiv:2306.12338*.

- Jesse Dodge, Gabriel Ilharco, Roy Schwartz, Ali Farhadi, Hannaneh Hajishirzi, and Noah Smith. 2020. Fine-tuning pretrained language models: Weight initializations, data orders, and early stopping. *arXiv preprint arXiv:2002.06305*.
- Finale Doshi-Velez and Been Kim. 2017. Towards a rigorous science of interpretable machine learning. *arXiv preprint arXiv:1702.08608*.
- Thomas Durieux, João F. Ferreira, Rui Abreu, and Pedro Cruz. 2020. Empirical review of automated analysis tools on 47,587 Ethereum smart contracts. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE)*, pages 530–541.
- Matt Gardner, Yoav Artzi, Victoria Basmov, Jonathan Berant, Ben Bogin, Sihao Chen, Pradeep Dasigi, Dheeru Dua, Yanai Elazar, Ananth Gottumukkala, et al. 2020. Evaluating models’ local decision boundaries via contrast sets. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1307–1323.
- Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. 2021. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92.
- Fabrizio Gilardi, Meysam Alizadeh, and Maël Kubli. 2023. ChatGPT outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences*, 120(30):e2305016120.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.
- Namgyu Ho, Laura Schmid, and Se-Young Yun. 2023. Large language models are reasoning teachers. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14852–14882.
- Cheng-Yu Hsieh, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alexander Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. 2023. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 8003–8017.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.
- Alon Jacovi and Yoav Goldberg. 2020. Towards faithfully interpretable NLP systems: How should we define and evaluate faithfulness? In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4198–4205.
- Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, et al. 2023. Measuring faithfulness in chain-of-thought reasoning. *arXiv preprint arXiv:2307.13702*.
- Liunian Harold Li, Jack Hessel, Youngjae Yu, Xiang Ren, Kai-Wei Chang, and Yejin Choi. 2023. Symbolic chain-of-thought distillation: Small models can also “think” step-by-step. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2665–2679.
- Lucie Charlotte Magister, Jonathan Mallinson, Jakub Adamek, Eric Malmi, and Aliaksei Severyn. 2023. Teaching small language models to reason. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 1773–1781.
- Curtis Northcutt, Lu Jiang, and Isaac Chuang. 2021. Confident learning: Estimating uncertainty in dataset labels. *Journal of Artificial Intelligence Research*, 70:1373–1411.
- Qwen Team. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- Dalila Ressi, Alvis Spanò, Matteo Rizzo, Lorenzo Benetollo, and Sabina Rossi. 2026. Reentrancy detection in the age of LLMs. *arXiv preprint arXiv:2603.26497*.
- Francesco Salzano, Cosmo Kevin Antenucci, Simone Scalabrino, Giovanni Rosa, Rocco Oliveto, and Remo Pareschi. 2026. An empirical analysis of vulnerability detection tools for solidity smart contracts. *Empirical Software Engineering*, 31:143.
- Emma Strubell, Ananya Ganesh, and Andrew McCallum. 2019. Energy and policy considerations for deep learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3645–3650.
- Yuqiang Sun, Daoyuan Wu, Yue Xue, Han Liu, Haijun Wang, Zhengzi Xu, Xiaofei Xie, and Yang Liu. 2024. GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*.
- Alex Warstadt, Alicia Parrish, Haokun Liu, Anhad Mohananey, Wei Peng, Sheng-Fu Wang, and Samuel R. Bowman. 2020. BLiMP: The benchmark of linguistic minimal pairs for English. *Transactions of the Association for Computational Linguistics*, 8:377–392.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35.

- Sarah Wiegreffe, Ana Marasović, and Noah A. Smith. 2021. Measuring association between labels and free-text rationales. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 10266–10284.
- Chavhan Sujeet Yashavant, MitrajSinh Chavda, Saurabh Kumar, Amey Karkare, and Angshuman Karmakar. 2024. SCRUBD: Smart contracts reentrancy and unhandled exceptions vulnerability dataset. *arXiv preprint arXiv:2412.09935*.
- Zibin Zheng, Jianzhong Su, Jiachi Chen, David Lo, Zhijie Zhong, and Mingxi Ye. 2024. DAppSCAN: Building large-scale datasets for smart contract weaknesses in DApp projects. *IEEE Transactions on Software Engineering*, 50(6).
- Zibin Zheng, Neng Zhang, Jianzhong Su, Zhijie Zhong, Mingxi Ye, and Jiachi Chen. 2023. Turn the rudder: A beacon of reentrancy detection for smart contracts on Ethereum. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*.